

OPTIMIZING BASEBALL

Oliwier Krawczyk 284208

May - June 2026

1 Introduction - Problem Definition

The Problem is inspired by the movie "Moneyball", where the main character played by Brad Pitt uses mathematics and statistics to optimize his baseball team roster. This translates to a Multiple-Choice Knapsack model, in which we have to add budget and cardinality constraints.

2 Mathematical Formulation

2.1 Decision Variables

Binary variables determining whether a player is selected for the roster. Let N_B be the total number of batters and N_P be the total number of pitchers:

$$x_i \in \{0, 1\} \quad \text{for each batter } i \text{ in the database, } i = 1, 2, \dots, N_B \quad (1)$$

$$y_j \in \{0, 1\} \quad \text{for each pitcher } j \text{ in the database, } j = 1, 2, \dots, N_P \quad (2)$$

2.2 The Objective Function

Maximize the total projected utility (Value) of the team. Advanced sabermetric metrics will be used as the expected value (V_i for batters and V_j for pitchers):

$$\max \left(\sum_{i=1}^{N_B} V_i x_i + \sum_{j=1}^{N_P} V_j y_j \right) \quad (3)$$

2.3 Constraints

The model incorporates both classic resource limits and specific business (sports) dependencies:

Budget Constraint (Salary Cap): The sum of the costs (c_i, c_j) of the selected batters and pitchers cannot exceed the set budget B :

$$\sum_{i=1}^{N_B} c_i x_i + \sum_{j=1}^{N_P} c_j y_j \leq B \quad (4)$$

Positional Constraints (Structural Requirements): The roster must precisely cover the required field positions (e.g., exactly 1 Catcher, 3 Outfielders, 1 Starting Pitcher, etc.). Let P_k represent the subset of batters eligible for a specific position k :

$$\sum_{i \in P_k} x_i = \text{Required_amount} \quad (5)$$

$$\sum_{j=1}^{N_P} y_j = 1 \quad (\text{Exactly 1 Starting Pitcher}) \quad (6)$$

3 Bacterial Foraging Optimization (BFO) - Pseudocode

To solve the problem metaheuristically, the BFO algorithm was implemented and tuned using the Optuna framework.

Algorithm 1 BFO Metaheuristic for Roster Optimization

```
1: Initialize  $S$  bacteria (random valid rosters) and set  $health = 0$ 
2: for  $ell = 1$  to  $N_{ed}$  do ▷ Elimination-Dispersal Loop
3:   for  $k = 1$  to  $N_{re}$  do ▷ Reproduction Loop
4:     for  $j = 1$  to  $N_c$  do ▷ Chemotaxis Loop
5:       for  $i = 1$  to  $S$  do
6:         Tumble: Generate neighbor roster  $x'$  by swapping one player
7:         if  $fitness(x') > fitness(x_i)$  then
8:            $x_i \leftarrow x'$ 
9:         Swim: Continue moving in same direction up to  $N_s$  times if improving
10:        end if
11:        Update  $health_i \leftarrow health_i + fitness(x_i)$ 
12:        Update Global Best Roster if  $fitness(x_i)$  is highest and within budget
13:      end for
14:    end for
15:    Reproduction: Sort bacteria by health. Eliminate bottom  $S/2$ . Duplicate top  $S/2$ .
16:  end for
17:  Dispersal: For each bacterium, replace with random roster with probability  $P_{ed}$ .
18: end for
19: return Global Best Roster
```

4 Complexity and Results

4.1 Computational Complexity

MILP (Gurobi Solver): Since the Multiple-Choice Knapsack Problem is NP-hard, the exact solver relies on the Branch and Bound algorithm.

- **Time Complexity:** $\mathcal{O}(2^N)$ in the worst-case scenario, as the search tree can expand exponentially.
- **Space (Memory) Complexity:** $\mathcal{O}(2^N)$ in the worst-case, required to store active nodes in the tree.

BFO Metaheuristic: The metaheuristic approach provides a scalable alternative with fixed iteration limits.

- **Time Complexity:** The total number of fitness evaluations is constant and bounded by hyperparameters $(S \cdot N_{ed} \cdot N_{re} \cdot N_c \cdot N_s)$. However, since neighbor generation scales with the database size, the overall time complexity is linear $\mathcal{O}(N)$, safely avoiding the exponential explosion of exact methods.
- **Space (Memory) Complexity:** $\mathcal{O}(N + S)$, where S is the population size, and N is required to store the initial player database. Memory usage grows only linearly, preventing any out-of-memory bottlenecks.

4.2 Performance Comparison

Table 1: Performance and Optimal Rosters Comparison: Gurobi vs. BFO

Approach	Total Value	Used Budget	Optimum Gap	Execution Time
Gurobi (MILP)	713.17	\$49,402	0.00%	Moderate (Exact)
BFO (Optuna)	695.46	\$49,056	2.48%	Fast (Heuristic)

Position	Gurobi (MILP Exact)	BFO (Metaheuristic)
Starting Pitcher (SP)	Pitcher_72 (\$6207)	Pitcher_72 (\$6207)
Catcher (C)	Batter_101 (\$4697)	Batter_35 (\$6865)
First Base (1B)	Batter_246 (\$4609)	Batter_246 (\$4609)
Second Base (2B)	Batter_54 (\$4599)	Batter_54 (\$4599)
Third Base (3B)	Batter_243 (\$4844)	Batter_243 (\$4844)
Shortstop (SS)	Batter_90 (\$8445)	Batter_185 (\$5931)
Outfielder (OF)	Batter_1 (\$4589)	Batter_1 (\$4589)
Outfielder (OF)	Batter_124 (\$6168)	Batter_124 (\$6168)
Outfielder (OF)	Batter_231 (\$5244)	Batter_231 (\$5244)

The BFO algorithm successfully identified 7 out of 9 identical players compared to the global optimum, settling in a high-quality local maximum. This proves the metaheuristic’s efficiency, achieving 97.52% of the mathematical maximum while maintaining significantly lower computational and memory complexity.